

Genel Ön Bilgi

Linked List yapısında her Node temel olarak data ve next alanlarından oluşur:

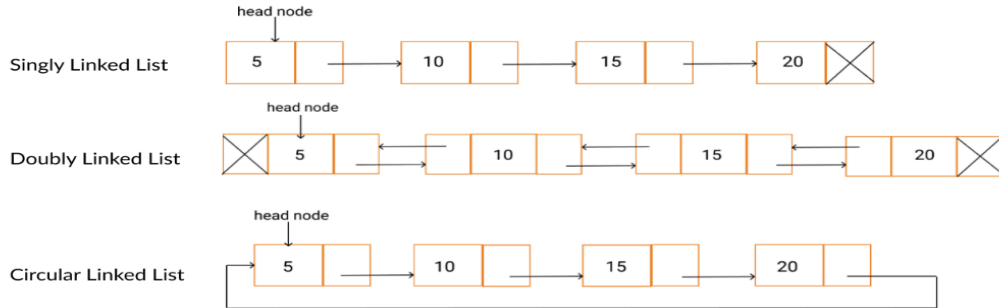
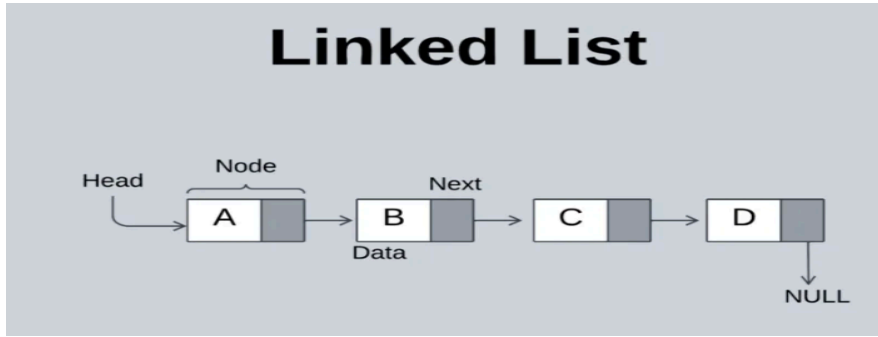
```
struct Node {  
    int data;  
    struct Node *next;  
};
```

Temel yapı:

```
+-----+-----+  
| data | next | -----> sonraki Node  
+-----+-----+
```

Örnek:

[10 | *] □ [20 | *] □ [30 | NULL]



SORU 1 – Tek Node Oluşturma

Soru

Bir Node yapısı oluşturun. Node içerisinde 10 değerini saklayın ve ekrana yazdırın.

Çizim / Mantık

```
+-----+-----+  
| 10 | NULL |  
+-----+-----+
```

Çözüm

```
#include <stdio.h>  
  
struct node  
{  
    int data;  
    struct node *next;  
};  
  
int main()  
{  
    struct node dugum = {10,NULL};  
    printf("%d", dugum.data);  
}
```

Not

Öğrenilecek: struct, data, next, NULL.

SORU 2 – İki Node'u Birbirine Bağlama

Soru

10 ve 20 değerlerini içeren iki Node oluşturun ve 10 → 20 → NULL listesini oluşturun.

Çözüm

```
#include <stdio.h>

struct node
{
    int data;
    struct node *next;
};

int main()
{
    struct node dugum1 = {20, NULL};
    struct node dugum0 = {10, &dugum1};
    printf("%d", dugum0.next->data);
}
```

Not

Önemli: node1.next-->data, node1'in sonraki Node'unun data alanına erişir.

SORU 3 – Üç Node'lu Listeyi Yazdırma

Soru

10 □ 20 □ 30 □ NULL listesini oluşturun ve while kullanarak tüm elemanları yazdırın.

Çizim / Mantık

head

|

v

[10] □ [20] □ [30] □ NULL

Çözüm

```
#include <stdio.h>

struct node
{
    int data;
    struct node *next;
};

int main()
{
    struct node dugum2 = {30, NULL};
    struct node dugum1 = {20, &dugum2};
    struct node dugum0 = {10, &dugum1};
    struct node *current = &dugum0;
    while (current != NULL)
    {
        printf("%d ", current->data);
        current = current->next;
    }
}
```

Not

Temel traversal mantığı: `current = current->next;`

SORU 4 – malloc Kullanarak Linked List Oluşturma

Soru

malloc() kullanarak 10 --> 20 --> 30 --> NULL listesini oluşturun.

Çizim / Mantık

head □ [10] □ [20] □ [30] □ NULL

Çözüm

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

int main()
{
    struct node *dugum0 = (struct node*)malloc(sizeof(struct node));
    struct node *dugum1 = (struct node*)malloc(sizeof(struct node));
    struct node *dugum2 = (struct node*)malloc(sizeof(struct node));

    dugum0->data = 10;
    dugum0->next = dugum1;

    dugum1->data = 20;
    dugum1->next = dugum2;

    dugum2->data = 30;
    dugum2->next = NULL;

    printf("%d %d %d", dugum0->data,dugum1->data,dugum2->data);

    free(dugum0); free(dugum1); free(dugum2);
}
```

Not

malloc ile bellek ayırdığımız için kullanım sonunda free() önemlidir.

SORU 5 – Listenin Eleman Sayısını Bulma

Soru

10 □ 20 □ 30 □ 40 □ NULL listesinin kaç Node içerdiğini bulun.

Mantık

[10] □ [20] □ [30] □ [40] --> NULL
1 2 3 4

Çözüm

```
#include <stdio.h>

struct node
{
    int data;
    struct node *next;
};

int main()
{
    struct node dugum3 = {40, NULL};
    struct node dugum2 = {30, &dugum3};
    struct node dugum1 = {20, &dugum2};
    struct node dugum0 = {10, &dugum1};
    struct node *current = &dugum0;
    int count = 0;
    while (current != NULL)
    {
        count++;
        printf("%d. %d\n", count, current->data);
        current = current->next;
    }
    printf("\nTotal Nodes = %d", count);
}
```

Not

Her Node ziyaret edildiğinde count bir artırılır.

SORU 6 – Listenin Elemanlarını Toplama

Soru

10 □ 20 □ 30 □ 40 □ NULL listesindeki değerlerin toplamını bulun.

Çizim / Mantık

sum = 0

10 □ 10

20 □ 30

30 □ 60

40 □ 100

Çözüm

```
#include <stdio.h>

struct node
{
    int data;
    struct node *next;
};

int main()
{
    struct node dugum3 = {40, NULL};
    struct node dugum2 = {30, &dugum3};
    struct node dugum1 = {20, &dugum2};
    struct node dugum0 = {10, &dugum1};
    struct node *current = &dugum0;
    int sum = 0;
    while (current != NULL)
    {
        printf("%d ", current->data);
        sum += current->data;
        current = current->next;
    }
    printf("\nTotal data = %d", sum);
}
```

Not

Traversal sırasında sum değişkenine current-->data eklenir.

SORU 7 – Linked List'te Eleman Arama

Soru

Kullanıcının girdiği sayının 10 □20 □30 □40 □NULL listesinde olup olmadığını bulun.

Çizim / Mantık

10 == aranan? Hayır

20 == aranan? Hayır

30 == aranan? Evet (örnek)

Çözüm

```
#include <stdio.h>

struct node
{
    int data;
    struct node *next;
};

int main()
{
    int input;
    printf("Please enter the number to check if it's matching.\n");
    scanf("%d",&input);
    struct node dugum3 = {40, NULL};
    struct node dugum2 = {30, &dugum3};
    struct node dugum1 = {20, &dugum2};
    struct node dugum0 = {10, &dugum1};
    struct node *current = &dugum0;
    while (current != NULL)
    {
        if (current->data == input)
        {
            printf("\nThere is a \"%d\" matching in the linked list!", current->data);
            return 0;
        }
        current = current->next;
    }
    printf("\nThere's no matching!");
}
```

Not

Eleman bulununca break ile arama sonlandırılabilir.

SORU 8 – Listenin Başına Eleman Ekleme

Soru

10 □ 20 □ 30 □ NULL listesinin başına 5 ekleyin. Sonuç 5 □ 10 □ 20 □ 30 □ NULL olmalıdır.

Çizim / Mantık

Başlangıç:

head □ [10] □ [20] □ [30] □ NULL

Yeni:

newNode □ [5]

Bağla:

[5] □ [10] □ [20] □ [30] □ NULL

Çözüm

```
#include <stdio.h>

struct node
{
    int data;
    struct node *next;
};

int main()
{
    struct node dugum3 = {40, NULL};
    struct node dugum2 = {30, &dugum3};
    struct node dugum1 = {20, &dugum2};
    struct node dugum0 = {10, &dugum1};
    struct node *head = &dugum0;

    struct node *current = head;
    while (current != NULL)
    {
        printf("%d ", current->data);
        current = current->next;
    }

    struct node newNode = {5, &dugum0};
    head = &newNode;

    current = head;
    printf("\n");
    while (current != NULL)
    {
        printf("%d ", current->data);
        current = current->next;
    }
}
```

Not

Baş eklemenin kritik iki satırı: newNode□next = head; ve head = newNode;

SORU 9 – Listenin Sonuna Eleman Ekleme

Soru

10 □ 20 □ 30 □ NULL listesinin sonuna 40 ekleyin.

Çizim / Mantık

Başlangıç:

[10] □ [20] □ [30] □ NULL

Sonra:

[10] □ [20] □ [30] □ [40] □ NULL

Çözüm

```
#include <stdio.h>

struct node
{
    int data;
    struct node *next;
};

int main()
{
    struct node dugum3 = {40, NULL};
    struct node dugum2 = {30, &dugum3};
    struct node dugum1 = {20, &dugum2};
    struct node dugum0 = {10, &dugum1};
    struct node *head = &dugum0;

    struct node *current = head;
    while (current != NULL)
    {
        printf("%d ", current->data);
        current = current->next;
    }

    struct node newNode = {50, NULL};

    current = head;
    printf("\n");
    while (current != NULL)
    {
        printf("%d ", current->data);
        if (current->next == NULL)
        {
            current->next = &newNode;
            printf("%d", current->next->data);
            break;
        }
        current = current->next;
    }
}
```

Not

Son Node'u bulmak için `while (current->next != NULL)` kullanılır.

SORU 10 – Belirli Bir Elemanı Silme

Soru

10 □ 20 □ 30 □ 40 □ NULL listesinden kullanıcının girdiği değeri silin.

Çizim / Mantık

Örnek:

[10] □ [20] □ [30] □ [40] □ NULL

30 silinirse:

[10] □ [20] □ [40] □ NULL
X [30]

Çözüm

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

int main()
{
    int input;
    printf("Please enter the number to check if it's matching: ");
    scanf("%d", &input);

    struct node *dugum0 = (struct node*)malloc(sizeof(struct node));
    struct node *dugum1 = (struct node*)malloc(sizeof(struct node));
    struct node *dugum2 = (struct node*)malloc(sizeof(struct node));
    struct node *dugum3 = (struct node*)malloc(sizeof(struct node));

    dugum0->data = 10; dugum0->next = dugum1;
    dugum1->data = 20; dugum1->next = dugum2;
    dugum2->data = 30; dugum2->next = dugum3;
    dugum3->data = 40; dugum3->next = NULL;

    struct node *head = dugum0;
    struct node *current = head;
    struct node *prev = NULL;
    int found = 0;

    while (current != NULL)
    {
        if (current->data == input)
        {
            found = 1;

            if (prev == NULL)
            {
```

```

        head = current->next;
    }
    else
    {
        prev->next = current->next;
    }

    free(current);
    printf("Node with value %d deleted successfully.\n\n", input);
    break;
}

prev = current;
current = current->next;
}

if (!found)
{
    printf("\nThere's no matching!\n\n");
}

printf("Updated list: ");
current = head;
while (current != NULL)
{
    struct node *temp = current;
    printf("%d ", current->data);
    current = current->next;
    free(temp);
}
}

```

Not

Silmede temel mantık: previous-->next = current-->next; ardından free(current). Head'in silinmesi ayrı ele alınır.

Fonksiyonlaştırılmış Linked List

Başlıca fonksiyonlar: insertBeginning(), display() ve search() fonksiyonları oluşturup;

- İlk nodun tuttuğu değeri getiren fonksiyon □ insertBeginning()
- Tüm listeyi gösteren fonksiyon □ display()
- Elaman ara □ search()

Çözüm

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

void insertBeginning(struct node *head)
{
    if (head != NULL)
    {
        printf("First node data: %d\n", head->data);
    }
    else
    {
        printf("List is empty!\n");
    }
}

void display(struct node *head)
{
    struct node *current = head;
    while (current != NULL)
    {
        printf("%d ", current->data);
        current = current->next;
    }
    printf("\n");
}

void search(struct node *head, int input)
{
    struct node *current = head;
    while (current != NULL)
    {
        if (current->data == input)
        {
            printf("Node with value %d found.\n", input);
            return;
        }
        current = current->next;
    }
    printf("There's no matching!\n");
}
```

```
int main()
{
    struct node *dugum0 = (struct node*)malloc(sizeof(struct node));
    struct node *dugum1 = (struct node*)malloc(sizeof(struct node));
    struct node *dugum2 = (struct node*)malloc(sizeof(struct node));
    struct node *dugum3 = (struct node*)malloc(sizeof(struct node));

    dugum0->data = 10; dugum0->next = dugum1;
    dugum1->data = 20; dugum1->next = dugum2;
    dugum2->data = 30; dugum2->next = dugum3;
    dugum3->data = 40; dugum3->next = NULL;

    struct node *head = dugum0;

    insertBeginning(head);

    printf("Updated list: ");
    display(head);

    search(head, 30);
    search(head, 99);

    struct node *current = head;
    while (current != NULL)
    {
        struct node *temp = current;
        current = current->next;
        free(temp);
    }

    return 0;
}
```